

CSA0503-DATABASE MANAGEMENT SYSTEMS

CO2 AT3

Debugging & Optimisation Task

1. Identify and fix the error in the given SQL query that uses incorrect JOIN syntax and returns duplicate results.

➤ **Incorrect Query:**

```
SELECT S.RegNo, S.Name, D.DeptName
FROM STUDENT S, DEPARTMENT D
INNER JOIN COURSE C
ON S.DeptID = D.DeptID
WHERE S.RegNo = C.RegNo;
```

➤ **Error Identification:**

- The query uses mixed JOIN syntax (comma join and INNER JOIN).
- Missing proper relationship condition between all tables.
- It may produce duplicate records due to incorrect table combination.

➤ **Corrected Query:**

```
SELECT S.RegNo, S.Name, D.DeptName
FROM STUDENT S
INNER JOIN DEPARTMENT D
ON S.DeptID = D.DeptID
INNER JOIN COURSE C
ON S.RegNo = C.RegNo;
```

➤ **Explanation:**

- Explicit JOIN syntax is used for better readability and performance.
- Proper join conditions prevent unwanted duplicate rows.
- The optimizer can execute the query more efficiently.

2. The following sub-query returns incorrect results due to a missing correlation. Debug and rewrite it correctly.

➤ **Incorrect Query:**

```
SELECT S.RegNo, S.Name, S.Salary
FROM STUDENT S
WHERE S.Salary >
(
    SELECT AVG(Salary)
    FROM STUDENT
);
```

➤ **Error Identification:**

- The sub-query calculates the average salary of the entire table.
- It is not correlated with the outer query.
- The query cannot compare each student's salary with their respective department average.

➤ **Corrected Query:**

```
SELECT S.RegNo, S.Name, S.Salary, S.DeptID
FROM STUDENT S
WHERE S.Salary >
(
    SELECT AVG(S1.Salary)
    FROM STUDENT S1
    WHERE S1.DeptID = S.DeptID
);
```

➤ **Explanation:**

- The sub-query is correlated using DeptID.
- Each student's salary is compared with the average salary of their own department.

3. A given SQL view definition causes errors when queried. Identify the issue and rewrite a correct and optimized view.

➤ **Incorrect Query:**

```
CREATE VIEW Student_Department_View AS  
  
SELECT S.RegNo, S.Name, D.DeptName, C.CourseDesc  
  
FROM STUDENT S, DEPARTMENT D  
  
WHERE S.DeptID = D.DeptID  
  
GROUP BY S.RegNo;
```

➤ **Error Identification:**

- The view contains a GROUP BY clause without using aggregate functions.
- All selected columns are not included in the GROUP BY clause.
- The old comma join syntax reduces readability and may cause query errors.

➤ **Corrected Query:**

```
CREATE VIEW Student_Department_View AS  
  
SELECT S.RegNo, S.Name,  
       D.DeptName, C.CourseDesc  
  
FROM STUDENT S  
  
INNER JOIN DEPARTMENT D  
ON S.DeptID = D.DeptID  
  
INNER JOIN COURSE C  
ON S.CourseNo = C.CourseNo;
```

➤ **Explanation:**

- The unnecessary GROUP BY clause is removed.
- Proper INNER JOIN syntax improves query clarity and performance.
- The view now correctly displays student, department, and course details without errors.

4. Debug the given relational algebra expression that produces incorrect output for a natural join operation.

➤ **Incorrect Query:**

STUDENT $\bowtie$ DEPARTMENT

ON STUDENT.RegNo = DEPARTMENT.DeptID

➤ **Error Identification:**

- The join condition uses incorrect attributes.
- RegNo represents the student register number, while DeptID represents department identification.
- The expression compares unrelated attributes, producing incorrect output.

➤ **Corrected Query:**

STUDENT $\bowtie$ DEPARTMENT

ON STUDENT.DeptID = DEPARTMENT.DeptID

➤ **Optimized Expression:**

π RegNo, Name, DeptName

(

STUDENT $\bowtie$ STUDENT.DeptID = DEPARTMENT.DeptID DEPARTMENT

)

➤ **Explanation:**

- The correct common attribute DeptID is used for the natural join.
- Projection (π) is applied to retrieve only required attributes.
- This reduces unnecessary data processing and improves query efficiency.

5. The SQL query below violates referential integrity. Identify the violation and modify the statements to enforce proper constraints.

➤ **Incorrect Query:**

```
CREATE TABLE STUDENT (  
    RegNo INT PRIMARY KEY,  
    Name VARCHAR(20),  
    DeptID INT  
);  
  
INSERT INTO STUDENT  
VALUES (101, 'Rahul', 10);
```

➤ **Error Identification:**

- The STUDENT table contains DeptID, but no foreign key constraint is defined.
- A student record can be inserted with a department ID that does not exist in the DEPARTMENT table.
- This violates referential integrity between related tables.

➤ **Corrected Query:**

```
CREATE TABLE STUDENT (  
    RegNo INT PRIMARY KEY,  
    Name VARCHAR(20),  
    DeptID INT,  
    FOREIGN KEY (DeptID)  
    REFERENCES DEPARTMENT(DeptID)  
);  
  
INSERT INTO STUDENT  
VALUES (101, 'Rahul', 1);
```

➤ **Explanation:**

- Invalid department values cannot be inserted.
- Referential integrity maintains consistency between parent and child tables.

6. Given an inefficient SQL query with nested loops, rewrite it using JOIN to optimize performance.

➤ **Incorrect Query:**

```
SELECT Name
FROM STUDENT
WHERE DeptID IN
(
    SELECT DeptID
    FROM DEPARTMENT
    WHERE DeptName = 'CSE'
);
```

➤ **Error Identification:**

- The query uses a nested sub-query, which may be executed repeatedly.
- It is less efficient for large tables.
- Using JOIN improves readability and query performance.

➤ **Corrected Query:**

```
SELECT S.RegNo, S.Name, D.DeptName
FROM STUDENT S
INNER JOIN DEPARTMENT D
ON S.DeptID = D.DeptID
WHERE D.DeptName = 'CSE';
```

➤ **Explanation:**

- The nested sub-query is replaced with an INNER JOIN.
- The JOIN directly matches records from both tables using DeptID.
- This reduces query execution time and improves performance, especially for large databases.

**7. The GROUP BY query below produces wrong aggregation results.
Debug the query and explain the error.**

➤ **Incorrect Query:**

```
SELECT DeptID, Name, COUNT(RegNo) AS TotalStudents  
FROM STUDENT  
GROUP BY DeptID;
```

➤ **Error Identification:**

- The column Name is selected but is **not included** in the GROUP BY clause.
- Name is also **not used with an aggregate function**.
- This results in an error (or unpredictable results in some SQL modes).

➤ **Corrected Query:**

```
SELECT DeptID,  
       COUNT(RegNo) AS TotalStudents  
FROM STUDENT  
GROUP BY DeptID;
```

➤ **Explanation:**

- Only the grouped column (DeptID) and the aggregated column (COUNT(RegNo)) are selected.
- The query now correctly counts the number of students in each department.
- The result contains one row per department with the total student count.

8. Identify and fix the logical error in a given SQL UPDATE statement that unintentionally modifies all rows in the table.

➤ **Incorrect Query:**

```
UPDATE STUDENT  
SET City = 'Chennai',  
    MobileNo = '9876543210'  
WHERE City = City;
```

➤ **Error Identification:**

- The condition WHERE City = City is always **TRUE**.
- As a result, **every row** in the STUDENT table is updated.
- This unintentionally modifies all student records.

➤ **Corrected Query:**

```
UPDATE STUDENT  
SET City = 'Chennai',  
    MobileNo = '9876543210'  
WHERE RegNo = 101;
```

➤ **Explanation:**

- The WHERE clause now specifies a unique student using RegNo.
- Only the intended record is updated.
- Using a primary key (or another specific condition) prevents accidental updates to all rows.

9. Optimize the given multi-table SQL query by restructuring sub-queries into JOINS and using appropriate indexes.

➤ **Incorrect Query:**

```
SELECT S.RegNo, S.Name
FROM STUDENT S
WHERE S.DeptID IN
(
    SELECT DeptID
    FROM DEPARTMENT
    WHERE DeptName = 'CSE'
)
AND S.CourseNo IN
(
    SELECT CourseNo
    FROM COURSE
    WHERE CourseType = 'C'
);
```

➤ **Error Identification:**

- The query uses multiple nested sub-queries, which increase execution time.
- The database has to evaluate each sub-query before filtering the STUDENT table.
- This approach is less efficient for large datasets.

➤ **Corrected Query:**

```
SELECT S.RegNo, S.Name,  
       D.DeptName, C.CourseDesc  
FROM STUDENT S  
INNER JOIN DEPARTMENT D  
ON S.DeptID = D.DeptID  
INNER JOIN COURSE C  
ON S.CourseNo = C.CourseNo  
WHERE D.DeptName = 'CSE'  
AND C.CourseType = 'C';
```

➤ **Explanation:**

- The nested sub-queries are replaced with INNER JOINS for faster data retrieval.
- Indexes on DeptID, CourseNo, DeptName, and CourseType speed up joins and filtering.
- This optimized query improves readability, reduces execution time, and performs efficiently on large databases.

10. Debug a given relational calculus expression that returns an empty result set due to incorrect quantifier usage.

➤ **Incorrect Query:**

$$\{ S.Name \mid STUDENT(S) \wedge \\ \forall D (DEPARTMENT(D) \wedge \\ S.DeptID = D.DeptID \wedge \\ D.DeptName = 'CSE') \}$$

➤ **Error Identification:**

- The expression uses the **universal quantifier ($\forall$)** incorrectly.
- It requires every department in the DEPARTMENT table to have the name 'CSE', which is impossible.
- As a result, the query returns an **empty result set**.

➤ **Corrected Query:**

$$\{ S.Name \mid STUDENT(S) \wedge \\ \exists D (DEPARTMENT(D) \wedge \\ S.DeptID = D.DeptID \wedge \\ D.DeptName = 'CSE') \}$$

➤ **Explanation:**

- Replace the universal quantifier ($\forall$) with the existential quantifier ($\exists$).
- The corrected expression returns students who belong to **at least one department** named 'CSE'.
- Using the correct quantifier ensures accurate and meaningful query results.